

Navigators — Complete Engineering Audit & Validation Prompt

Context:

This repository contains the Navigators project, an Intelligent Dead Reckoning (IDR) system designed to solve the SIH26168 problem statement. The documentation claims a very specific, high-performance architecture. However, the documentation does NOT prove that the current codebase actually implements and validates these claims.

Your Objective:

Audit the actual repository first, prove what exists by running it, identify every gap between implementation and the hard requirements, then fix those gaps and validate every claimed capability with measurable tests. Never fabricate a metric or mark a component complete merely because the documentation says it is complete.

Hard Requirements to Audit & Validate

1. 100% Offline Edge Execution

The AI model, Extended Kalman Filter (EKF), and navigation engine MUST run on-device, in JavaScript, with ZERO backend/internet/cellular dependencies.

- **Audit Task:** Prove that the JS PWA can run the entire inference and navigation loop while completely offline. Any FastAPI/WebSocket backend can *only* be treated as a dev/test tool.

2. 15-State Extended Kalman Filter (EKF) in JavaScript

The EKF requirement is strict. It must be a full 15-state filter (Position, Velocity, Orientation, Accel Bias, Gyro Bias) fusing AI velocity and GNSS directly on-device.

- **Audit Task:** If the current `ekf.js` or `offline_engine.js` is simplified compared to the Python reference, that is a major gap. Port, implement, and mathematically validate the full 15-state logic in JavaScript.

3. TCN/LSTM 2D Velocity Prediction

The AI output must be treated as 2D velocity (or higher), not just scalar speed.

- **Audit Task:** Verify that the TCN/LSTM actually predicts 2D velocity from windowed IMU data. Do not quietly reduce the problem to scalar speed unless benchmark validation proves that design is sufficient. Check the ONNX export and edge integration.

4. Automatic Phone-to-Vehicle Alignment

Phone-to-vehicle alignment is a core component, not an optional enhancement. Automatic pitch/roll/yaw alignment is explicitly required to translate raw IMU frames to the vehicle frame.

- **Audit Task:** Verify the mathematics of the coordinate-frame transformations. Write tests across different phone orientations to prove the alignment logic works.

5. Non-Linear Signal Filtering

- **Audit Task:** Ensure the preprocessor strictly implements non-linear median filtering + Butterworth low-pass filtering to strip impulsive shocks, potholes, bumps, and phone movement *before* data hits the AI.

6. Correct Navigation Pipeline Flow

NHC, ZUPT, and Map Matching belong strictly in the navigation pipeline, not as UI post-processing.

- **Audit Task:** Validate the pipeline strictly follows this flow:
 - **GNSS Available:** GNSS + IMU → AI → EKF → map matching
 - **GNSS Lost:** IMU → AI → Dead Reckoning → NHC/ZUPT → map matching
 - **GNSS Returns:** Dead Reckoning → Reacquisition → GNSS + INS fusion

7. Measurable Performance Targets

Do not assume drift targets are met. You must prove them.

- < 10% positional drift.
- < 5 m drift over 50 m GNSS-denied travel.
- < 100 m drift over 1 km at 60 km/h.
- 10 Hz smartphone GNSS+INS loop execution.
- ≥ 200 Hz FOG-IMU capability on the edge engine.

Execution Workflow

You must follow this exact sequence:

1. **AUDIT:** Read the core code (ekf.js, matrix.js, offline_engine.js, tcn_model.py, preprocessor.py).
2. **REPRODUCE:** Run the existing tests and benchmarks (scripts/benchmark.py).
3. **DIAGNOSE:** Identify all gaps between the current implementation and the hard requirements listed above.
4. **IMPLEMENT:** Fix the gaps. Write the missing JS code, fix the 15-state EKF, enforce 2D velocity, fix the alignment math.
5. **BENCHMARK & VALIDATE:** Run measurable tests to prove the fixes meet the SIH performance targets.
6. **ITERATE:** Do not stop until the system mathematically proves its capabilities offline.